
Lecture 1: What is firmware?

The basics of C/C++ programming and firmware design.

Robojackets Firmware Training 2026

Who am I?

Jaxon Parriott

- 4th Year CompE BS
- Robocup Firmware Lead
- ECE 2031 TA
- GTNH Noob

It's software's fault



Literally anything else



Who am I?

Sean Seguin

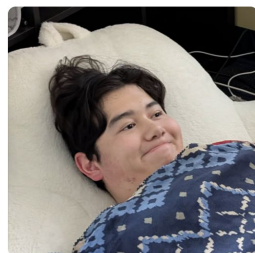
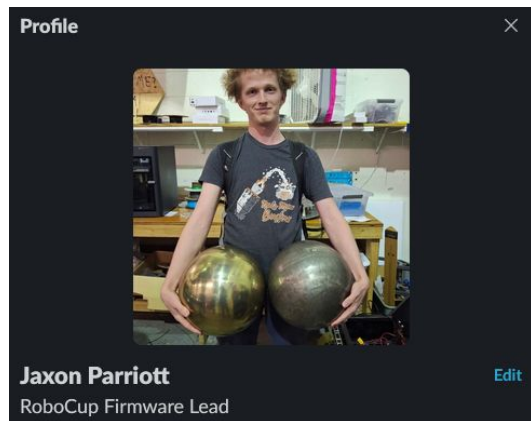
- 3rd Year CompE BS/MS
- Robocup Firmware Engineer
- Power Board Lead
- ECE 2031 TA
- Deadlock Enjoyer
- Autostore AI/CV



I ripped my hand open
building this thing to
hold our cameras



Administrivia - Slack



Sean M. V. Seguin

ECE | Firmware Engineer | Robocup Firmware |
AI/CV@Autostore
lol

- Join the RoboJackets Slack at robojackets.slack.com
- Set up your Slack profile with:
 - ◆ Some photo of yourself
 - ◆ Your subteam/positions
- Join important channels:
 - ◆ #firmware-training-helpdesk
 - ◆ #<team> (e.g. #robocup)
 - ◆ #<team>-firmware (e.g. #robocup-firmware)
 - ◆ #<team>-electrical (e.g. #robocup-electrical)

Administrivia - Logistics

- Trainings are held twice a week, both sessions are identical
 - ◆ Thursday from 6 PM - 8 PM @ Skiles 271
 - ◆ Friday from 6 PM - 8 PM @ Skiles 271 (EXCEPT WEEK ONE!!! 171)
 - Soldering training will be held at the Hive Makerspace and replaces the lecture for that week
 - ◆ Session 1: Monday, 09/28 from 6:30 PM - 8:30 PM
 - ◆ Session 2: Thursday, 10/01 from 6:30 PM - 8:30 PM
 - Any changes from above dates/times will be announced via Slack in #firmware-training-helpdesk
 - GO TO ELECTRICAL TRAINING!!!
 - ◆ Not mandatory but very very very very useful as a firmware developer to understand the electrical side!
 - ◆ Monday from 5 PM - 8 PM @ Skiles 270
 - ◆ Thursday from 5 PM - 8 PM @ Van Leer C340
-

Administrivia - Training Outline

- **Lecture 1: C/C++ Basics (today)**
 - Lecture 2: Interrupts & Analog I/O
 - Lecture 3: Communication Protocols
 - Lecture 4: Driver Development
 - Lecture 5: State Machines & RTOS
 - Practical Soldering Training
-

Administrivia - AI Usage

These labs are simple, AI can almost definitely finish the entire thing in just one prompt. You will successfully learn nothing more than what lines of code look like.

Good AI Usage

- Slightly better Google
- Debugging
- Filtering through documents
- First pass code reviews
- Code snippet generation

Bad AI Usage

- Writing entire files
- Reformatting anything
- Final pass reviews
- Most things firmware related

While it has improved in recent years, most AI is still terrible at writing firmware.
This is why you exist. Never copy AI code unchecked!

What is firmware?



Firmware

The bridge between silicon and sensors

- Firmware is the bare metal software embedded in almost every modern electronic device.
 - Even computers utilize firmware for BIOS, sub devices (i.e. mice, speakers, GPUs), memory, etc.
 - All firmware is software but not all software is firmware!
 - Firmware is a very specialized subset of software designed to run on a specific device, to do a specific thing, and that is it. As well, it deals with much lower level device management.
 - Most Common Languages
 - Assembly
 - C/C++
 - Rust
 - Python
-

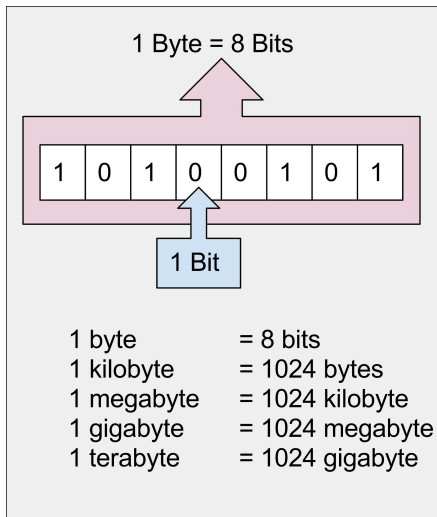
What is code?

Binary

All information on any device is stored as binary!

Computers can only differentiate between on and off, so all data must be based on that concept

1 1 0 0 1 0 1 0
↓
 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0



Bitwise Operator Truth Table

X	Y	X&Y	X Y	X^Y	~(X)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Different Ways to Use Binary

1101 1110

OR (|)

1000 0001

=

1101 1111

1101 1110

XOR (^)

1000 0001

=

0101 1111

1101 1110

AND (&)

1000 0001

=

1000 0000

1101 1110

NOT (~)

=

0010 0001

1101 1110

+

1000 0001

=

(1) 0101 1111

!Overflow!

2's Complement
To Add Negatives

-34

+

-1

|

V

0010 0010 (34)

+

0000 0001 (1)

|

V

1101 1101

+

0010 0011 (35)

^

1111 1110

|

0010 0010

V

^

1101 1110

+

1101 1101 = 1111 1111

Data Types

While all information is stored as binary, data types allow you to ensure that the information is correctly interpreted.

- The most fundamental data types are the integer data types.

- char
- short
- int
- long
- float
- double
- bool

```
3 // char
4 uint8_t x = 0; // 1 byte | 0 - 255
5 int8_t y = 0; // 1 byte | -128 - 127
6
7 // short
8 uint16_t x = 0; // 2 bytes | 0 - 65,535
9 int16_t y = 0; // 2 bytes | -32,768 - 32,767
10
11 // int
12 uint32_t x = 0; // 4 bytes | 0 - 4,294,967,295
13 int32_t y = 0; // 4 bytes | -2,147,483,648 - 2,147,483,647
14
15 // long long
16 uint64_t x = 0; // 8 bytes | 0 - 18,446,744,073,709,551,615
17 int64_t y = 0; // 8 bytes | -9,223,372,036,854,775,808 - 9,223,372,036,854,775,807
18
19 float x = 0; // 4 bytes | -3.4 × 1038 - 3.4 × 1038 | 7 place precision
20 double y = 0; // 8 bytes | -1.7 × 10308 - 1.7 × 10308 | 15 place precision
21
22 bool condition = false; // 1 byte | true or false | In c/c++ 0 == false, not 0 == true
```

- Signed integer data types use one bit to denote sign.
-

Boolean Examples

Boolean expressions are used to test whether or not a statement evaluates to true

```
// true and false are defined as expected
bool a = true != false; // TRUE
// Any numeric value is equal to true except 0
bool b = 1; // TRUE
uint8_t num = 0;
bool c = num; // FALSE
// Boolean expressions can be used to combine logical statements
bool d = a && c; // FALSE
bool e = a || c; // TRUE
bool f = b ^ e; // FALSE
bool g = !f; // TRUE
// Most common comparators are ==, !=, >, <, >=, <=
bool h = (12 == 12) && (12 != 3) && (32 > 31) || ((3 >= 3) && (4 < 31)); // TRUE
// Try and keep them tidy!
bool True = false;
bool False = true;
bool i = !(False && (True != (0 ^ 0))) || (("true" > "True") && (True ^ (!(0b1101 ^ 0b0010 ^ 0b1111) ^ ("ture??" > 'f')))); // TRUE!
```

Conditionals

Conditionals allow you to choose whether or not to run certain lines of code.

if-else

```
bool condition_1 = false;
uint8_t condition_2 = 15;

if (condition_1) {
    // Do something
} else if (condition_2) {
    // Do something else if
} else if (1 == 1 && 2 != 3) {
    // Do something else if
} else {
    // Do otherwise
}
```

switch-case

```
uint8_t var = 7;

switch (var) {
    case 0:
        // Do something
        break;
    case 7:
        // Do something
        break;
    case 10:
    case 11:
        // Do something for both 10 or 11
        break;
    default:
        // Do otherwise
}
```

Loops

While loops run “while” something is true.

```
bool condition = true;

while (condition) {
    // Loops as long as condition is true
}
```

```
bool condition = true;

do {
    // Runs at least once
    // Loops as long as condition is true
} while (condition);
```

For loops run “for” a certain amount of times.

```
// (var def; condition; function)
for (size_t i = 0; i < 10; i++) {
    // Loops until condition is true
    // Applies the function each loop
}
```

Exit loops at any time with “break” (only exits immediate loop).

Functions

Functions prevent code repetition and overall simplify code.

Functions must be unique:

- Different names same parameters?
 - Different functions
- Same names different parameters?
 - Overloaded function
- Same names same parameters?
 - Error

```
// Function definition tells your program the function exists
// return_type function_name(var_type, var_type, ...);
uint64_t add_numbers(uint64_t, uint64_t);

void setup() {

}

void loop() {

    // Function usage must match expected types
    uint64_t x = add_numbers(1, 1);
}

// Function implementation
uint64_t add_numbers(uint64_t num_1, uint64_t num_2) {
    // Additional code...
    return num_1 + num_2;
}
```

Pointers and Complex Data Structures

Pointers are not complex. Pointers just point to where a section of data is stored in memory.

```
uint8_t x = 15; // Actual stored value
uint8_t* a = &x; // Pointer to where x is stored in memory
*a = 12; // Dereference pointer (bring in x) and replace with 12
// Now x == 12
```

Arrays allow the unified storage of a lot of the same data type. (They are actually also pointers)

```
uint8_t arr1[10] = {}; // Initializes empty (0s)
uint8_t arr2[] = {1, 2, 3, 4, 5}; // Initializes full
uint8_t arr3[100]; // Uninitialized
// arr1[3] == 0
// arr2[3] == 4
// arr3[3] == ?
// *(arr2 + 3) == 4 Note: arrays are pointers!
```

Structs allow the combination of multiple data types

```
struct rgb_t {
    uint8_t red = 0; // Entries can be initialized
    uint8_t green = 0;
    uint8_t blue = 0;
    String name; // Or left uninitialized
    // Structs can have functions tied to them!
    uint32_t pack() {
        return ((red << 16) | (green << 8) | blue);
    }
    // Special function called a constructor!
    rgb_t(String name) : name(name) {
        red = 32; // Can assign values by macro or internal
    } // By default is blank ie rgb_t();
};
// Init with constructor
rgb_t color1("Blue");
// Multiple layouts all the same
rgb_t color2 = rgb_t("Green");
// Read/Write values with dot commands
color2.red; // Returns 32!
color2.green = 256;
// Or call functions
uint32_t bits = color1.pack();
// Use pointers to structs with ->
(&color1)->name; // Returns "Blue"!
```

Additional Data Structures

Enums can be used to create custom variables to give meaning to your code.

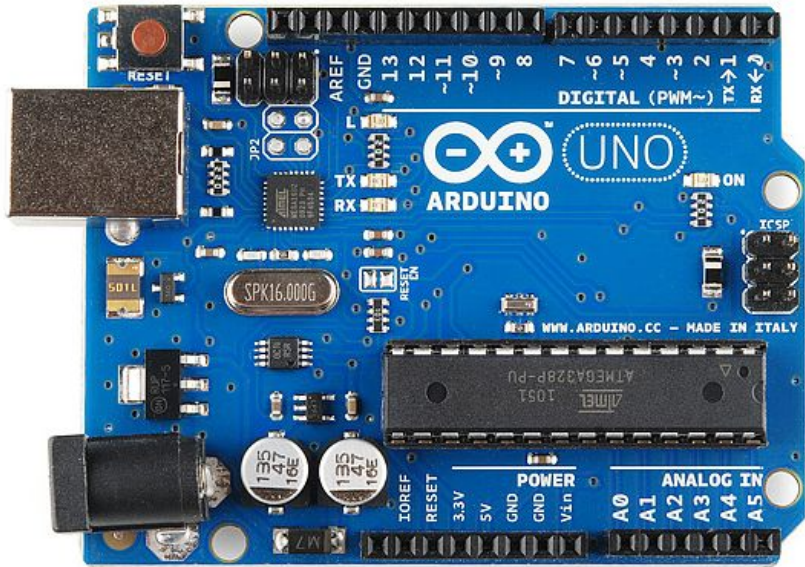
```
enum DeviceError {
    NoError = 0, // Can be assigned real integral values
    SensorError, // But not necessary
    DisplayError = "Bad", // Cannot be assigned just anything
    UnknownError,
};
// While type can be inferred, it is good practice to reinforce
DeviceError error = DeviceError::NoError;
// Useful for giving meaning to otherwise meaningless values
switch (error) {
    case SensorError:
        // Sensor error handling
        break;
    case DisplayError:
        // Display error handling
        break;
    // etc...
}
```

Classes are very similar to structs but allow for more control over access.

```
class Display {
    // Functions, vars, etc under public can be accessed by anything
public:
    Display(uint8_t address); // Constructor
    bool begin(uint64_t frequency); // Public functions
    bool set_screen(DisplayScreen screen);
    // Functions, vars, etc under private can only be used in the class itself
private:
    uint64_t _freq; // Private vars
    uint8_t _addr;
    void fast_send(); // Private functions
};
// Initialize with constructor
Display display = Display(0x16);
// Can call public functions
display.begin(2000000);
// But not private ones
display.fast_send();
uint8_t illegal_read = display._addr;
```

Arduino C++

What is arduino?



Arduino is an open source framework for writing code for embedded devices.

- Provides helper functions for interacting with the device's peripherals
- Provides a uniform structure to firmware

File Layout

- `#include <Arduino.h>`
 - Adds Arduino specific functions to your program
- `void setup() {}`
 - This is the program entry point
 - All code in this runs once at the start
- `void loop() {}`
 - This runs immediately after setup finishes
 - Loops forever

```
1  #include <Arduino.h>
2
3  // put function declarations here:
4  int myFunction(int, int);
5
6  void setup() {
7      // put your setup code here, to run once:
8      int result = myFunction(2, 3);
9  }
10
11 void loop() {
12     // put your main code here, to run repeatedly:
13 }
14
15 // put function definitions here:
16 int myFunction(int x, int y) {
17     return x + y;
18 }
```

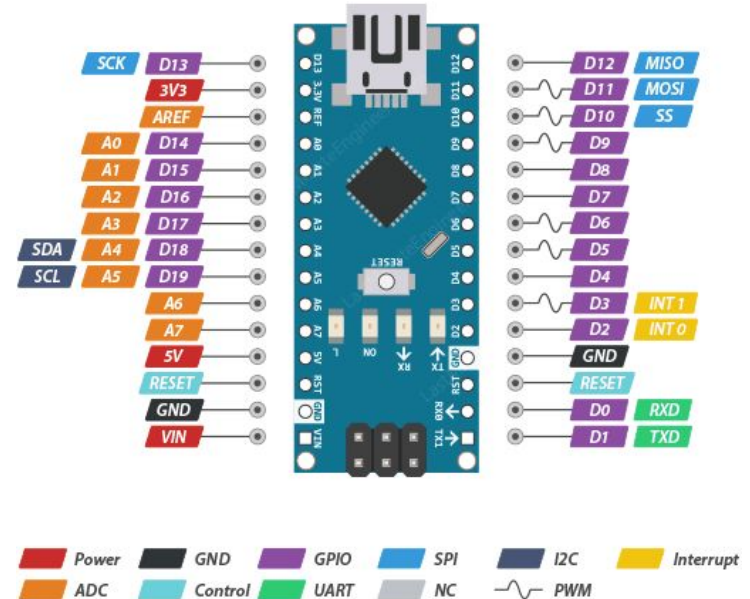
GPIO (General Purpose Input/Output)

GPIO are the pins on the microcontroller that you can assign uses to!

The most common uses include:

- Driving outputs
- Communication protocols
- Reading inputs

Be aware of what pins can do what! It differs per microcontroller!



Arduino Nano Pinout

Basic Arduino Functions

Arduino provides a plethora of functions to utilize the hardware and peripherals of the microcontroller. A few basic examples include:

- **delay(ms)**
 - Halts the program for however many ms
- **delayMicroseconds(us)**
 - Halts the program for however many us
- **millis()**
 - Returns how many ms has passed since power on.
- **pinMode(pin, mode)**
 - Initializes a pin to a mode (i.e. INPUT)
- **digitalWrite(pin, out)**
 - Drives a pin to “out”
- **digitalRead(pin)**
 - Reads the state of pin

There are many, many more than this which can be referenced here:

<https://docs.arduino.cc/language-reference/>

Lab Time!

Git Basics

Install Git Bash!

*Only Windows users

<https://git-scm.com/install/windows>

Windows and Linux:

```
cd ~/Documents
mkdir robojackets
cd robojackets
git clone https://github.com/RoboJackets/firmware-training.git
cd firmware-training
git checkout -b "yourgtusername-training"
```

Mac:

Similar process I hope

I also hope I have someone who can fill this out later

You will now have a local copy of the repo!

Common Git Commands

- **git init**
 - Starts a new git repo in the current directory
- **git clone <url> <*file>**
 - Copies a remote repo to local
 - Optional output file name
- **git add <*>**
 - Adds file(s) to current commit
 - To add all files use “.”
 - Required before committing
- **git commit -m “message”**
 - Creates a new commit (save point) with a message
- **git push**
 - Sends changes of local repo to remote
- **git pull**
 - Updates local repo from remote
- **git checkout <-b> <branch>**
 - Switches branches
 - Add “-b” to create new branch
- **git merge <branch>**
 - Combines “branch” with current branch
 - Have fun with merge conflicts!
- **git restore <*>**
 - Resets given files to last commit
 - To reset all files use “.”
- **git remote add origin <url>**
 - Links local repo to remote repo

And so many more! Google is your friend in this case! Or: <https://git-scm.com/docs>

Pull Requests

Just pushing changes to main is not a good idea for large scale repos!

Pull requests (PRs) are requests to merge a branch with main.

To create one, push your BRANCH to the remote repo. Then, from the GitHub website you can open a PR in the “Pull Requests” tab of the repo.

Creating a PR then allows a repo admin to review your work at least one more time before merging.

VERY IMPORTANT FOR ERROR CATCHING!!!

VSCode

Installation

Go to: <https://code.visualstudio.com/>

Follow the instructions for your system's installation.

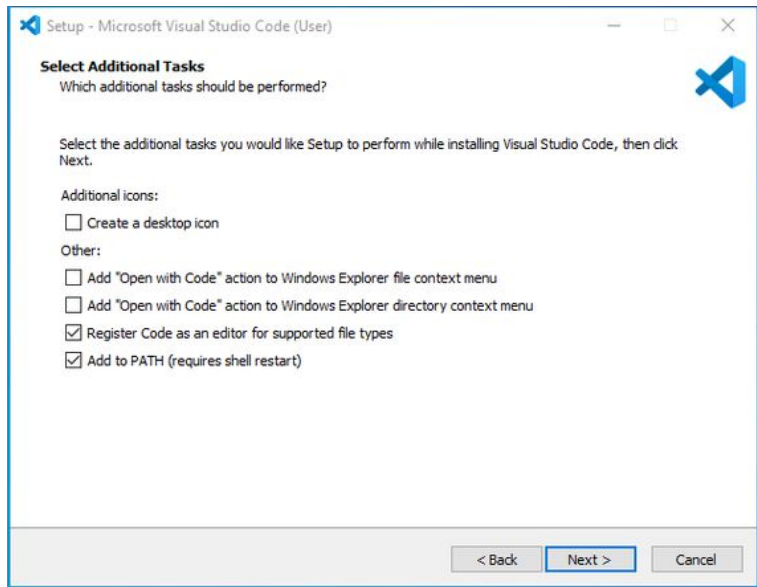
You are smart I believe in you B)

Windows (maybe Linux and Mac too?):

When you get to this specific screen →
Go ahead and tick all boxes for ease of use

!! Very Useful Shortcut !!

Since you will likely already have a terminal open for git, you can just run "code <*file>" to open a folder/file in vscode straight from terminal. No need to also open a file explorer! (if you are already in the root directory just run "code.")



Adding Extensions

Extensions can be downloaded from the extensions tab on the left!

There are many useful extensions for anything from code interpretation to automation.

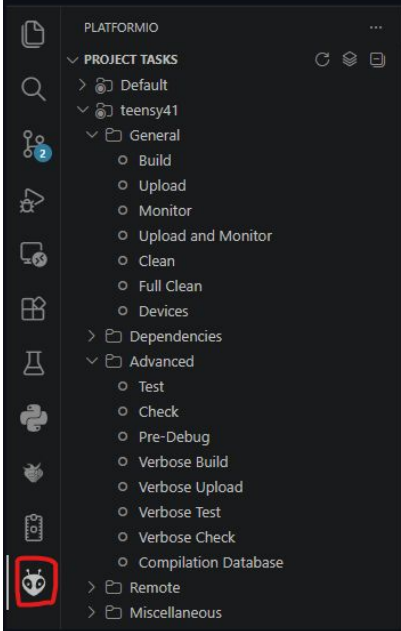
For the labs you are required to get:

- [Platformio](#)
 - Code builder for embedded programming
 - Should work out of the box
- [Wokwi](#)
 - Simulator for embedded devices
 - You have to sign up for a license as described in their documentation
 - (its free dw)



Using Your Extensions

- Build: Compiles your code but does not attempt to upload. Good for error checking.
- Upload: Compiles and attempts to upload to the teensy. May fail for many reasons but once compiled you can just press the program button on the teensy to try the upload again so long as you don't close the uploader.
- Clean: Removes all temporary files forcing a full recompile. Can fix many errors.
- Test: Runs the tests in the test folder. Requires the teensy to be available as they will be run on it.



← Platformio

For Wokwi:

Just click “Start Simulation” after compiling with PIO.

(Remember to recompile after each change!)

WOOO THE ACTUAL LAB

Blink the LED(s)!

Instructions are included in the lab files!

Assuming you still have your terminal open from the git section:

```
cd labs/week-1/blink  
code .
```

And begin!

If you need help let me or my TA know!
